

Industrial Software Development (ISD)

Academic Year 2026/2027

Instructors

Battista Biggio, Angelo Sotgiu

M.Sc. in Computer Engineering, Cybersecurity and Artificial Intelligence
University of Cagliari, Italy

Course Instructors



Battista Biggio - Full Professor
AI Security, PRA Lab



Angelo Sotgiu - Assistant Professor
AI Security, PRA Lab

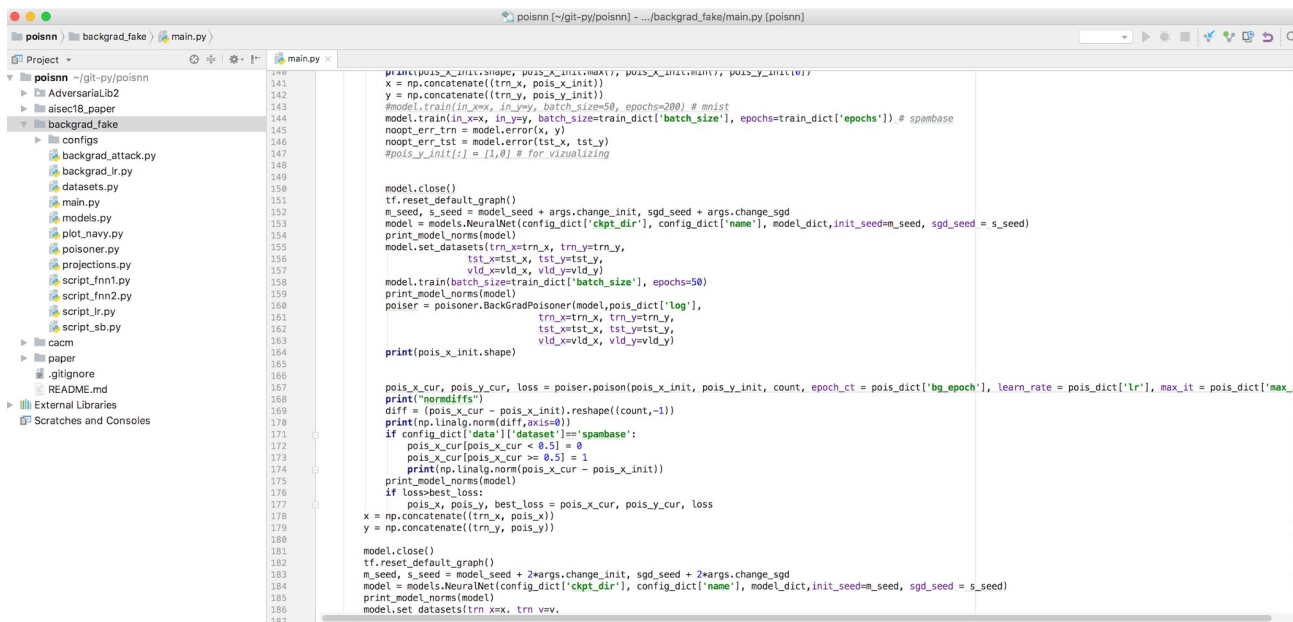


saifer Lab
Joint lab on Safety and Security of AI

www.saiferlab.ai

Why Industrial Software Development (ISD)?

- Software development as a (*young*) student / researcher is mostly learning and prototyping ideas in an unstructured manner



The screenshot shows a code editor window titled "poisnn [~/git-py/poisnn] - .../backgrad_fake/main.py [poisnn]". The left sidebar displays a project tree with the following structure:

- poisnn
├── AdversarialLib2
├── alicec18_paper
├── backgrad_fake
│ ├── configs
│ ├── backgrad_attack.py
│ ├── backgrad_lr.py
│ ├── datasets.py
│ ├── main.py
│ ├── models.py
│ ├── plot_navy.py
│ ├── poisoner.py
│ ├── projections.py
│ ├── script_fn1.py
│ ├── script_fn2.py
│ ├── script_lr.py
│ └── script_sb.py
├── cacm
├── paper
├── .gitignore
├── README.md
├── External Libraries
└── Scratches and Consoles

The main editor area displays the content of "main.py" (lines 140-187). The code is a Python script for a neural network experiment, featuring the following key sections:

- Initialization (lines 140-148):** Concatenates training and validation data, sets batch size (50) and epochs (200), and initializes the model.
- Model Training (lines 150-158):** Resets the default graph, sets the seed, and trains the model using the "NeuralNet" class.
- Model Evaluation (lines 159-167):** Prints the model norms and sets the datasets for training and validation.
- Model Training (lines 168-176):** Trains the model for a specific batch size (50) and prints the model norms.
- Poisoner Initialization (lines 177-187):** Initializes the "Poisoner" class with the model, training data, and validation data.
- Model Training (lines 188-196):** Prints the model norms and sets the datasets for training and validation.

How Far Can We Get?



Image credit: <https://medium.com/@gaperton/software-managing-the-complexity-caff5c4964cf>

Why Industrial Software Development (ISD)?

- Complexity of modern, industrial and open-source software projects cannot be managed as an unstructured process
 - it requires significant planning and managing work before writing even a single line of code!
 - (... and we have not yet even considered interactions with customers / stakeholders ...)
- Some examples (in the area of computer vision, machine/deep learning and AI)
 - OpenCV <https://github.com/opencv/opencv>
 - Scikit-learn <https://github.com/scikit-learn/scikit-learn>
 - Tensorflow <https://github.com/tensorflow/tensorflow>
 - PyTorch <https://github.com/pytorch/pytorch>



Things are getting even more complex...

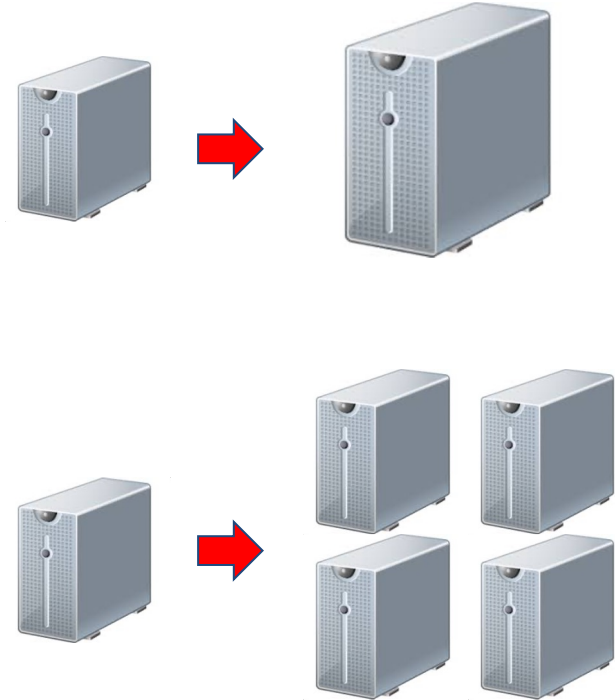
- AI has shown that very valuable information can be extracted from the vast amount of data available nowadays in many applications
- AI is helping but also making sw development more complex at the same time

Data created → **Data stored** → **Data analysis / AI** → **Actionable knowledge**



Vertical vs Horizontal Scalability

- **Vertical scalability:** increasing resources on a single machine - RAM, CPU, hard disk,...
 - no longer enough to cope with large data volumes and large predictive models (deep networks)
- **Horizontal scalability:** execute software and scale its execution on multiple machines, depending on the requested processing power
 - software has to be executed on heterogeneous architectures (e.g., cloud, data centers)



ISD: Main Lessons to be Learned

- Software can not be designed without following a systematized/managed procedure
 - It requires a massive dose of engineering and planning
 - Accounting for scalability / architectural requirements, etc.
- There cannot be improvisation. No need to reinvent the wheel
 - Be sure to adopt / re-use existing solutions to known problems!
- We will show you how to design software and algorithms in a principled manner
 - Software development / engineering is not just coding/programming!
 - AI tools must be correctly integrated into the development processes
- It is about defining a set of principles/values and a methodology to:
 - negotiate the required features that a piece of software should have with the customers,
 - create/update it using a well-defined strategy that accounts for cost and effort, and
 - plan its releases / updates in advance.

Why Industrial Software Development (ISD)?

- Learning and prototyping is different from developing industrial applications
 - Dealing with increasing complexity requires a standardized approach to managing projects
- **Software Development / Engineering:** set of tools and methodological approaches to plan, develop, test and maintain (complex) software projects

Goal of the Course

- **Software Development/ Engineering:** set of tools and methodological approaches to plan, develop, test and maintain complex software projects
 - Programming is not a trivial, unstructured process!
- **Methodologies**
 - Agile principles, tools and methodologies for software development/engineering
 - DevOps principles and basic aspects
 - AI-assisted development
- **Programming basics**
 - Object-oriented programming (OOP) abstractions
 - Architectural and design Patterns, APIs
 - Test-driven Development (TDD)
 - AI-based applications
- We will use *Python* as our main programming language in this course, but these principles apply to other languages as well!

Why Python?

- Pros
 - Easy to learn, interpreted language
 - Fast prototyping and execution
 - Many efficient third-party libraries (e.g., image processing, machine learning, etc.)
 - Wrappers to code written in different languages (mostly C/C++ libraries for efficient numerical computations - e.g., LAPACK)
- Cons
 - No strict control on types (sometimes causing unwanted behavior rather than raising error)
 - Memory management (not immediately clear when copying data or not...)

Good trade-off between efficiency and effort (no need to reinvent the wheel)!

Tentative Outline of the Course (7 CFU / 70 hours)

- Basics of (Python) Programming ~ 10 hours
- Version control (git) and collaborative development ~ 8 hours
- Foundations of industrial software development ~ 6 hours
- Test-Driven Development (TDD) ~ 6 hours
- CI/CD and containerization ~ 8 hours
- Software architecture and APIs ~ 6 hours
- AI-assisted development and AI-based systems ~ 22 hours
- Security in software development ~ 4 hours

What Are You Expected to Know?

- **Prerequisites**
 - Basic knowledge of object-oriented programming (OOP)
 - Basic knowledge / understanding of at least one programming language
 - Ability to write simple programs in Python
- No worries. We have prepared a small introduction to **Python** for this course, and dedicated tutoring sessions will start soon.
 - but please be ready to use it and practice during the course!

What Are You Expected to Learn?

- **Software development / engineering:** set of tools and methodological approaches to plan, develop, test and maintain complex software projects
 - Agile principles and methodologies
 - Test-driven development (TDD)
 - DevOps basics and Git
 - AI-assisted development
- **Programming**
 - Object-oriented programming (OOP) and abstractions
 - Architectural and design patterns, APIs
 - Code testing (using TDD as a methodological example)
 - AI-powered applications

Tools that Will Be Used in the Course

- PyCharm (IDE) with Python ≥ 3.12
 - The required Python libraries will be listed in the next lectures
- Git (for code versioning) - already integrated / supported in PyCharm
- Google Colab (Python notebook to run/edit code in your browser)
- Google Gemini free API Key (needs standard Google account)

How to Pass the Exam

- Home computer-exercise assignment (project) + written examination
 - Teams of 4 students maximum can do the home computer exercise
- Grading policy = Computer exercise (12/30) + written examination (20/30)
- **Written exam**
 - Questions with written answers